



Deep Learning

Dr. Shay Maymon

Regularization

A decorative graphic in the top right corner of the slide, featuring a network of interconnected nodes and lines, resembling a simplified neural network or a data flow diagram.

- **Goal in ML:** Perform well on **unseen data**, not just training data
- **Regularization:**

A set of techniques designed to **reduce the test error**
→ **May increase training error** intentionally

Dataset Augmentation in Object Recognition



Original



Inversion



Scaling



Translation



Rotation



Brightness



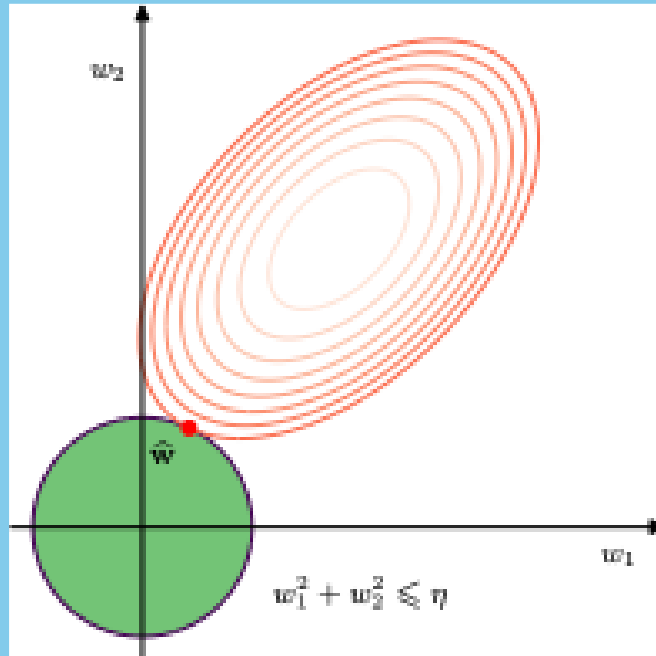
Additive Noise



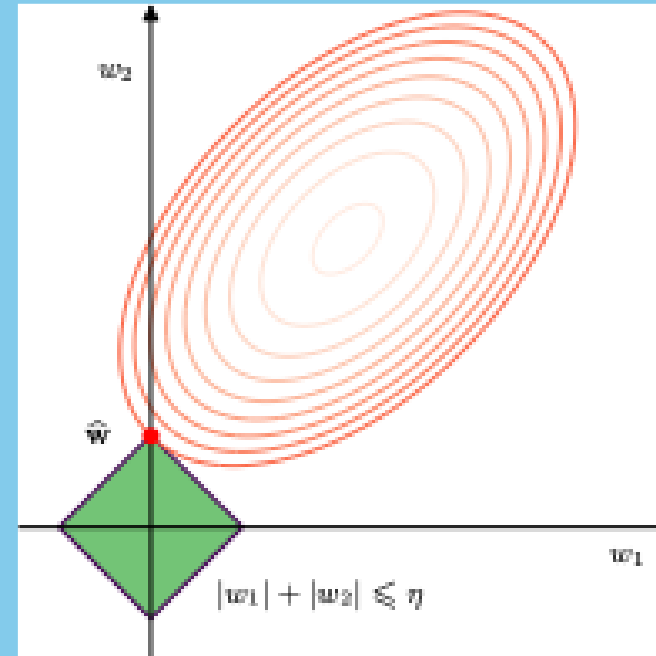
Color Shift

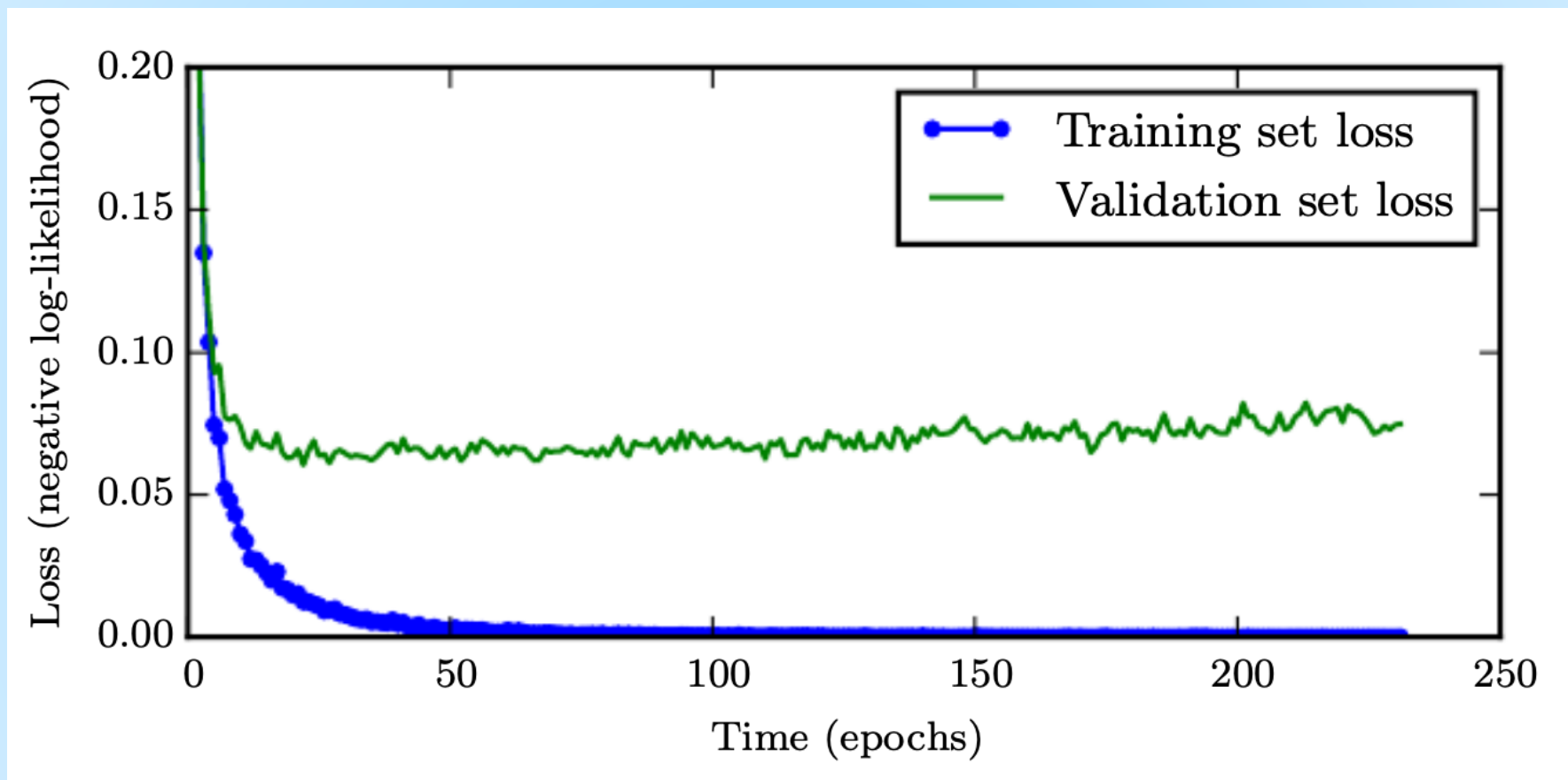
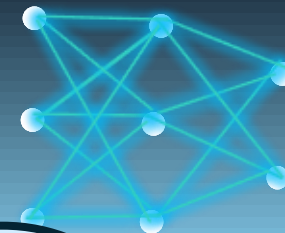
L^1 vs. L^2 Regularization: Sparsity Effect

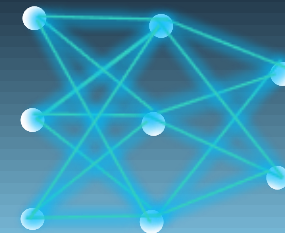
$$\tilde{J}(\mathbf{w}, X, \mathbf{y}) = J(\mathbf{w}, X, \mathbf{y}) + \frac{\alpha}{2} \|\mathbf{w}\|^2$$



$$\tilde{J}(\mathbf{w}, X, \mathbf{y}) = J(\mathbf{w}, X, \mathbf{y}) + \alpha \|\mathbf{w}\|_1$$



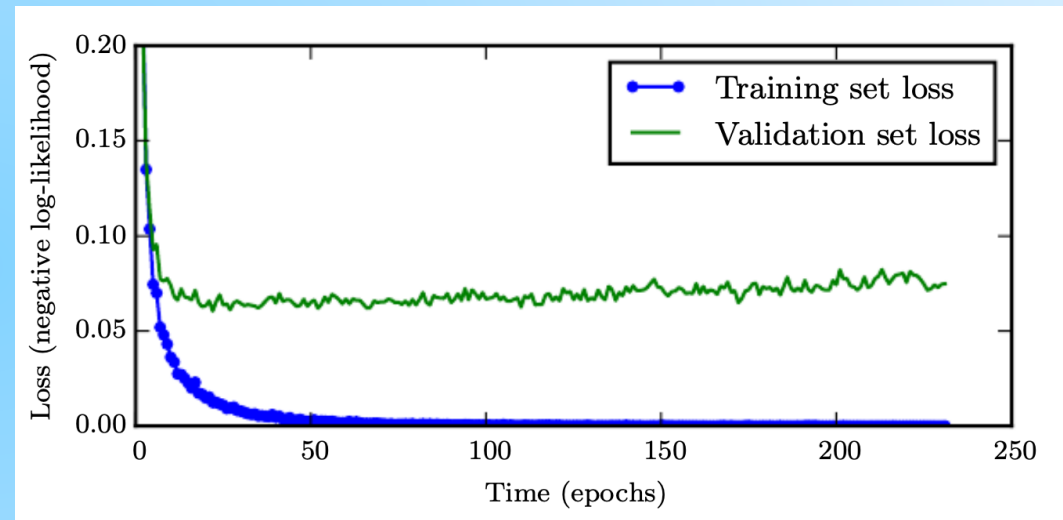




Early Stopping

Overfitting in Large Models

- **Large models can easily overfit**
High capacity allows them to memorize training data
- **Training error decreases steadily**
Model learns to fit the training set increasingly well
- **Validation error starts to increase**
Indicates poor generalization - model is fitting noise or spurious patterns
- **This behavior signals overfitting,**
and calls for regularization to control model complexity



Early Stopping: A Simple Regularization Strategy

A decorative graphic in the top right corner consisting of several blue dots connected by thin, glowing blue lines, resembling a neural network or a star map.

Stop training when validation performance stops improving

Early Stopping: A Simple Regularization Strategy



- **Track best validation error**

Save a copy of model parameters whenever validation error decreases

- **Final model = parameters with lowest validation error**

- The algorithm **terminates** when **no parameters have improved** over the best recorded validation error for some pre-specified number of iterations

- **Benefit:** Prevents overfitting by halting training before the model memorizes noise

Early Stopping: Efficient Hyperparameter Selection



- Early stopping is the **most commonly used** form of regularization in deep learning
- A very **efficient hyperparameter selection** algorithm
- The effective capacity of the model is controlled by determining the number of steps it can take to fit the training set

Early Stopping: A Simple Regularization Strategy

A decorative graphic in the top right corner consisting of several blue nodes connected by thin, glowing blue lines, resembling a neural network or a complex graph.

The **training time hyperparameter** is unique -
a single run of training tries out many values of the
hyperparameter

Early Stopping: Cost

- Cost
 - Running the validation set evaluation periodically during training
 - Ideally, this is done **in parallel** to the training process on a separate machine
 - These **periodic evaluations** may be reduced by
 - Using **a validation set that is small** compared to the training set
 - or by
 - **Evaluating the validation set error less frequently** and obtaining a lower resolution estimate of the optimal training time

Early Stopping: Cost



- Cost

- Need to maintain a copy of the best parameters

Store these parameters in a **slower and larger** form of **memory**. The best parameters are written to infrequently and **never read during training**, and thus these occasional slow writes have little effect on the total training time

Early Stopping

- Early stopping **requires almost no change** in the underlying training procedure, the objective function, or the set of allowable parameter values
- Early stopping may be used either **alone or in conjunction with other regularization strategies**

Early Stopping: Validation Set



- Early stopping **requires a validation set** – some training data is not fed to the model
- To **best exploit this extra data**, one can perform **extra training** after the initial training with early stopping has completed

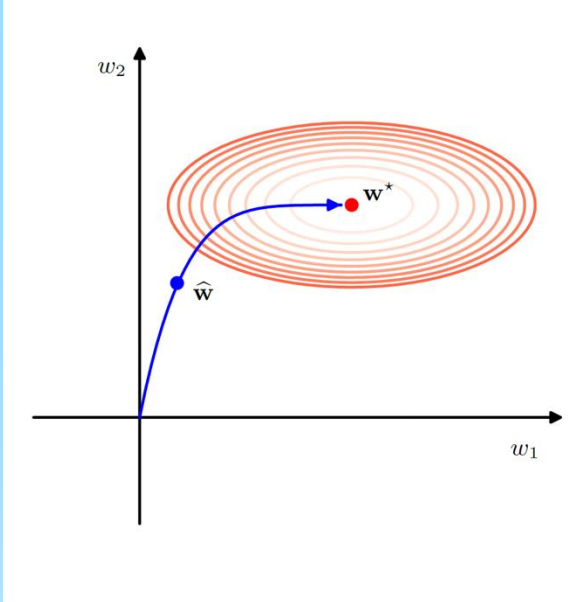
Early Stopping: Computational Cost



- Early stopping **reduces the computational cost** of the training procedure
 - Limiting the **number of training iterations**
 - Providing regularization **without requiring the addition of penalty terms** to the cost function or the computation of the gradients of such additional terms

Early stopping acts as a regularizer

- Early stopping has the effect of restricting the optimization procedure to a relatively small volume of parameter space in the neighborhood of the initial parameter value



Early stopping acts as a regularizer

- Imagining taking τ optimization steps (τ training iterations) with learning rate ε
- The product $\varepsilon\tau$ can be viewed as a measure of **effective capacity**
- Assuming the gradient is bounded, **restricting** both the **number of iterations** and **the learning rate** limits the **volume of parameter space** reachable from θ_0

Early stopping acts as a regularizer



We are going to study the **trajectory** followed by the parameter vector during training

A Quadratic Approximation

- Near a local minimum $\mathbf{w}^*$, the objective $J(\mathbf{w})$ can be approximated by:

$$\hat{J}(\mathbf{w}) = J(\mathbf{w}^*) + \frac{1}{2} (\mathbf{w} - \mathbf{w}^*)^T H (\mathbf{w} - \mathbf{w}^*)$$

- No first-order (gradient) term
- Hessian H is positive semidefinite

$$\nabla_{\mathbf{w}} \hat{J}(\mathbf{w}) = H (\mathbf{w} - \mathbf{w}^*)$$

Gradient Descent

- One gradient descent step :

$$\begin{aligned}\mathbf{w}^{(\tau)} &= \mathbf{w}^{(\tau-1)} - \varepsilon \nabla_{\mathbf{w}} \hat{J}(\mathbf{w}^{(\tau-1)}) \\ &= \mathbf{w}^{(\tau-1)} - \varepsilon \left(H(\mathbf{w}^{(\tau-1)} - \mathbf{w}^*) \right)\end{aligned}$$

$$\begin{aligned}\mathbf{w}^{(\tau)} - \mathbf{w}^* &= (\mathbf{w}^{(\tau-1)} - \mathbf{w}^*) - \varepsilon \left(H(\mathbf{w}^{(\tau-1)} - \mathbf{w}^*) \right) \\ &= (I - \varepsilon H)(\mathbf{w}^{(\tau-1)} - \mathbf{w}^*)\end{aligned}$$

Gradient Descent

$$\mathbf{w}^{(\tau)} - \mathbf{w}^* = (I - \varepsilon H)(\mathbf{w}^{(\tau-1)} - \mathbf{w}^*)$$

$$H = Q\Lambda Q^T$$

$$\begin{aligned}\mathbf{w}^{(\tau)} - \mathbf{w}^* &= (I - \varepsilon Q\Lambda Q^T)(\mathbf{w}^{(\tau-1)} - \mathbf{w}^*) \\ &= (QQ^T - \varepsilon Q\Lambda Q^T)(\mathbf{w}^{(\tau-1)} - \mathbf{w}^*) \\ &= Q(I - \varepsilon\Lambda)Q^T(\mathbf{w}^{(\tau-1)} - \mathbf{w}^*)\end{aligned}$$

Gradient Descent

$$\mathbf{w}^{(\tau)} - \mathbf{w}^* = Q(I - \varepsilon\Lambda)Q^T(\mathbf{w}^{(\tau-1)} - \mathbf{w}^*)$$

$$\begin{aligned} Q^T(\mathbf{w}^{(\tau)} - \mathbf{w}^*) &= (I - \varepsilon\Lambda)Q^T(\mathbf{w}^{(\tau-1)} - \mathbf{w}^*) \\ &= \dots = (I - \varepsilon\Lambda)^\tau Q^T(\mathbf{w}^{(0)} - \mathbf{w}^*) \end{aligned}$$

Gradient Descent

- Assume that $\mathbf{w}^{(0)} = \mathbf{0}$ and that ε is chosen to be small enough to guarantee $|1 - \varepsilon\lambda_i| < 1$,

$$Q^T(\mathbf{w}^{(\tau)} - \mathbf{w}^*) = (I - \varepsilon\Lambda)^\tau Q^T(\mathbf{w}^{(0)} - \mathbf{w}^*)$$

$$Q^T(\mathbf{w}^{(\tau)} - \mathbf{w}^*) = -(I - \varepsilon\Lambda)^\tau Q^T \mathbf{w}^*$$

$$Q^T \mathbf{w}^{(\tau)} = Q^T \mathbf{w}^* - (I - \varepsilon\Lambda)^\tau Q^T \mathbf{w}^* = (I - (I - \varepsilon\Lambda)^\tau) Q^T \mathbf{w}^*$$

L_2 Regularization - Weight Decay

$$\tilde{\mathbf{w}} = (H + \alpha I)^{-1} H \mathbf{w}^*$$

$$(Q^T \tilde{\mathbf{w}}) = (\Lambda + \alpha I)^{-1} \Lambda (Q^T \mathbf{w}^*)$$

$$(\Lambda + \alpha I)^{-1} (\Lambda + \alpha I) = I$$

$$(\Lambda + \alpha I)^{-1} \Lambda = I - (\Lambda + \alpha I)^{-1} \alpha$$

$$Q^T \tilde{\mathbf{w}} = (I - (\Lambda + \alpha I)^{-1} \alpha) Q^T \mathbf{w}^*$$

Early Stopping vs L_2 Regularization

$$Q^T \tilde{\mathbf{w}} = (I - (\Lambda + \alpha I)^{-1} \alpha) Q^T \mathbf{w}^*$$

$$Q^T \mathbf{w}^{(\tau)} = (I - (I - \varepsilon \Lambda)^\tau) Q^T \mathbf{w}^*$$

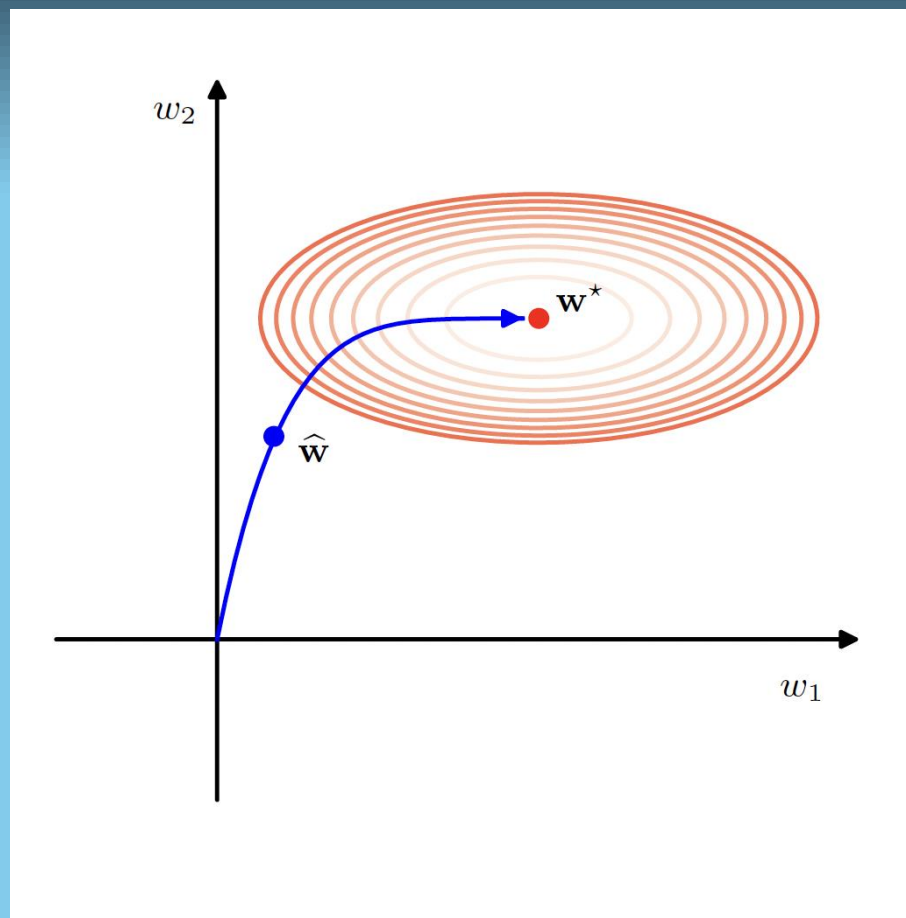
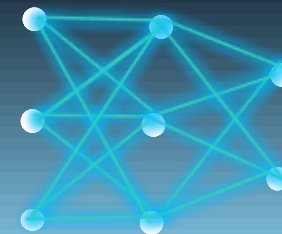
$$(I - \varepsilon \Lambda)^\tau = (\Lambda + \alpha I)^{-1} \alpha$$

$$(1 - \varepsilon \lambda_i)^\tau = \frac{\alpha}{\lambda_i + \alpha} = \frac{1}{1 + \lambda_i / \alpha}$$

$$\tau \log(1 - \varepsilon \lambda_i) = -\log(1 + \lambda_i / \alpha)$$

$$\tau(-\varepsilon \lambda_i) \approx -(\lambda_i / \alpha)$$

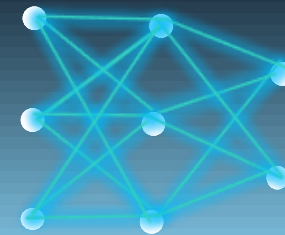
$$\tau \approx \frac{1}{\varepsilon \alpha}, \alpha \approx \frac{1}{\varepsilon \tau}$$



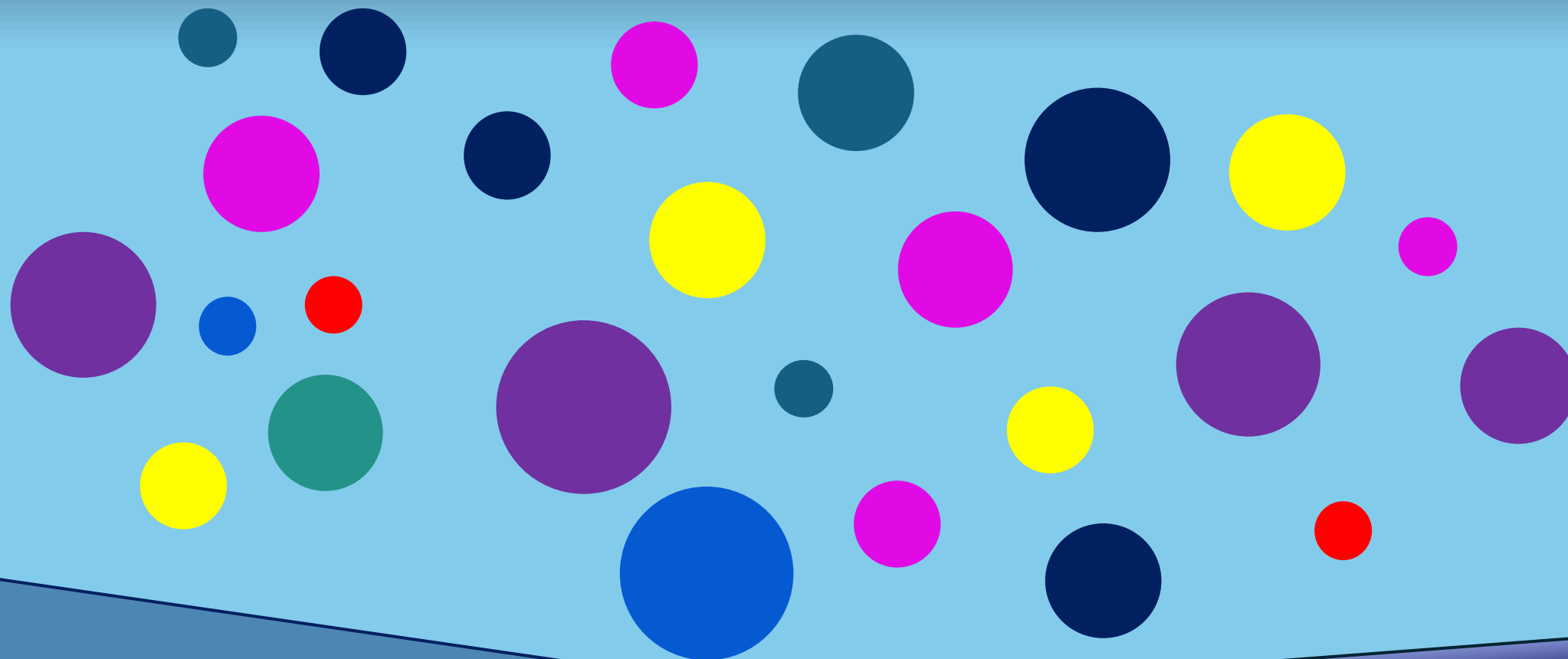
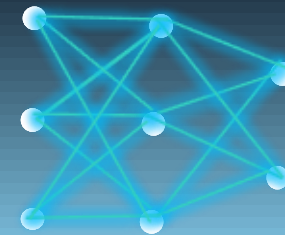
Early stopping acts as a regularizer



- Parameter values corresponding to **directions of significant curvature** are **regularized less** than **directions of less curvature**
- Parameters that correspond to **directions of significant curvature** tend to **learn early** relative to parameters corresponding to **directions of less curvature**



Ready?



Ensemble Methods

- Train multiple models independently
- Combine outputs at test time:
 - ✓ Classification: use **majority vote**
 - ✓ Regression: **average** predictions
- Benefit:
 - ✓ Reduces generalization error by combining several models



Ensemble Methods



- **Model averaging** improves generalization by combining multiple models
- Ensemble methods use this strategy
 - **Combine predictions** to reduce errors and variance
- **Why it works:**
 - Different models make **different errors**
 - **Averaging** reduces the chance of all models being wrong together
- **Result:** More **robust** and **accurate** predictions

A set of M Regression Models

- Consider a **regression problem** with an input $\mathbf{x}$ and a single output y
- Suppose we have a set of trained models and we form a committee prediction

$$y_{com}(\mathbf{x}) = \frac{1}{M} \sum_{m=1}^M y_m(\mathbf{x})$$

$$y_m(\mathbf{x}) = h(\mathbf{x}) + \varepsilon_m(\mathbf{x})$$

$$E(\varepsilon_m(\mathbf{x})) = 0$$

$$E(\varepsilon_m(\mathbf{x})^2) = \sigma^2$$

$$E(\varepsilon_m(\mathbf{x})\varepsilon_n(\mathbf{x})) = \rho\sigma^2, m \neq n$$

A set of M Regression Models

$$E \left((y_m(\mathbf{x}) - h(\mathbf{x}))^2 \right) = E(\varepsilon_m(\mathbf{x})^2) = \sigma^2$$

$$\begin{aligned} E \left((y_{com}(\mathbf{x}) - h(\mathbf{x}))^2 \right) &= E \left(\left(\left(\frac{1}{M} \sum_{m=1}^M y_m(\mathbf{x}) \right) - h(\mathbf{x}) \right)^2 \right) \\ &= E \left(\left(\frac{1}{M} \sum_{m=1}^M (y_m(\mathbf{x}) - h(\mathbf{x})) \right)^2 \right) = E \left(\left(\frac{1}{M} \sum_{m=1}^M \varepsilon_m(\mathbf{x}) \right)^2 \right) \end{aligned}$$

A set of M Regression Models

$$\begin{aligned} E \left((y_{com}(\mathbf{x}) - h(\mathbf{x}))^2 \right) &= E \left(\left(\frac{1}{M} \sum_{m=1}^M \varepsilon_m(\mathbf{x}) \right)^2 \right) \\ &= \left(\frac{1}{M} \right)^2 E \left(\sum_{m=1}^M \sum_{n=1}^M \varepsilon_m(\mathbf{x}) \varepsilon_n(\mathbf{x}) \right) \\ &= \frac{1}{M} \sigma^2 + \frac{M-1}{M} \rho \sigma^2 \\ &= \frac{1 + (M-1)\rho}{M} \sigma^2 \end{aligned}$$

Ensemble Error Reduction in Regression

- Ensemble MSE **depends on correlation**:
 - ✓ If errors are **perfectly correlated**:
 - **No benefit** from averaging
 - ✓ If errors are **uncorrelated**:
 - **Error decreases** linearly with ensemble size
- **Key Insight**:
 - ✓ Model averaging always helps or holds steady
 - ✓ When errors are independent, ensembles perform much better

Ensemble Methods: Different Paths to Model Diversity

A decorative graphic in the top right corner consisting of several blue nodes connected by thin, glowing blue lines, resembling a network or neural network structure.

- Ensemble methods **differ** in how they construct **individual models**
- **General approach:**
Train multiple models that make diverse errors, then combine them

Ensemble Methods: Different Paths to Model Diversity

A decorative graphic in the top right corner consisting of several blue nodes connected by thin, glowing blue lines, resembling a neural network or a data flow diagram.

- **Model diversity** can come from:
 - ✓ Using different model types
 - ✓ Changing training algorithms or objective functions

Ensemble Methods: Different Paths to Model Diversity

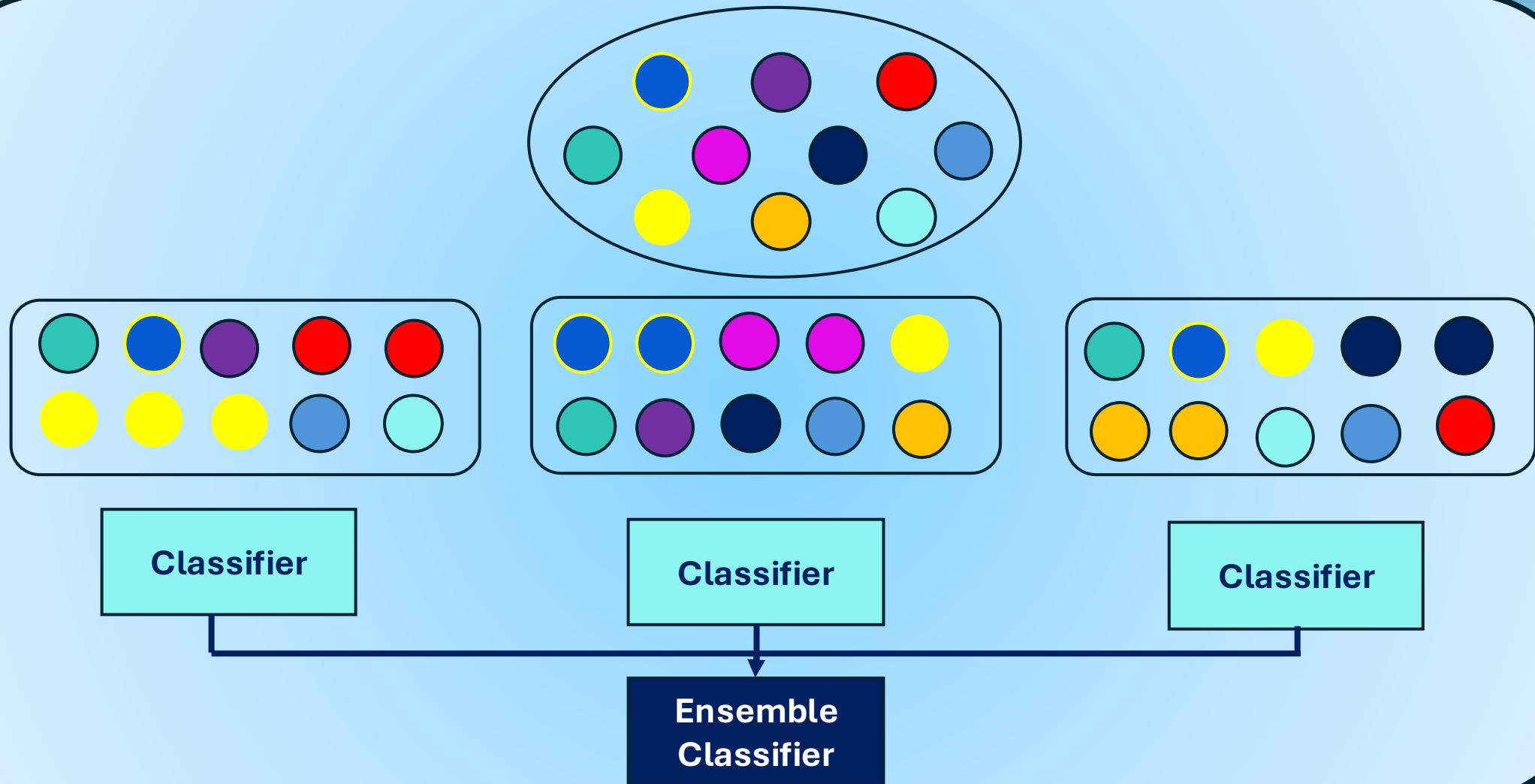


- **Bagging** (Bootstrap Aggregating) is a specific type ensemble method:
 - ✓ Uses the **same model type** and **training procedure**
 - ✓ Achieves diversity by training on k different bootstrapped datasets
- **Result:** Ensemble of similar models that differ due to data variation

Bagging

- Each model is trained on a different dataset, constructed as follows:
 - ✓ Sample with replacement from the original dataset
 - ✓ Each new dataset has the same size as the original
- This process creates k unique datasets
- Model i is trained on Dataset i
- Why this works:
Differences in sampled examples → lead to variations in learned models → Encourages ensemble diversity
- Final prediction: aggregate all models (e.g., majority vote or averaging)

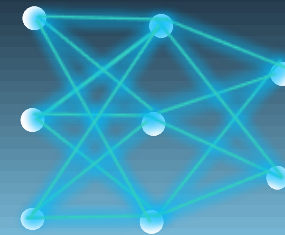
Bagging



Model Averaging



- Extremely **effective** at **reducing generalization error**
→ Often leads to **significant performance gains**
- **Trade-off**: Requires more **computation and memory**
→ Train and store **multiple models**
- **Benchmarking** caution:
 - ✓ Not typically used in scientific benchmarks
 - ✓ Because any algorithm can benefit, comparisons become unfair
- Use model averaging in practice, but avoid it in algorithm comparisons



Dropout

Dropout: Motivation and Impact



- **Deep neural networks** use multiple non-linear hidden layers :
 - ✓ highly **expressive**
 - ✓ prone to **overfitting** (can **capture patterns** that are **just noise**)
Especially **problematic** when:
 - ❖ Training data is **limited**
 - ❖ The **network** is **large** and **complex**
 - ✓ Such spurious patterns **do not generalize** to unseen test data, even if drawn from the same distribution. The result: **good performance** on **training** data, poor performance on **new data**

Why Traditional Ensembling is Impractical



- **Ensembling:**
 - ✓ Improves performance by **combining diverse models** (Different **architectures** or Different **training data**)
- **Ensemble** of multiple large networks:
 - ✓ computationally **expensive**. Each model needs **separate hyperparameter tuning**
 - ✓ There isn't **enough data** to train models on different subsets (Large networks **require lots of training data**)

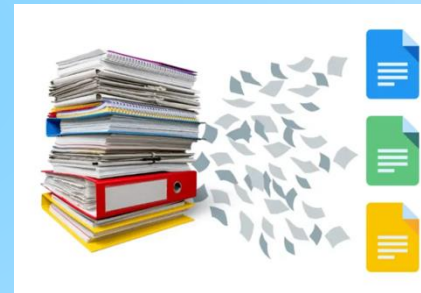
Why Traditional Ensembling is Infeasible at Test Time

A decorative graphic in the top right corner consisting of several blue nodes connected by thin, glowing blue lines, resembling a neural network or a data flow diagram.

- Even if **many large models** are trained: **Inefficient at test time**
 - ✓ Running them all at inference is **too slow**.
 - ✓ **Not practical for real-time** or latency-sensitive applications
- Need a method that gives **ensemble-like performance** with **minimal test-time cost**

Dropout: Motivation and Impact

- Dropout - an efficient regularization technique to combat overfitting
- Proven to significantly improve performance across multiple supervised learning tasks:



- Achieves state-of-the-art results on several benchmark datasets

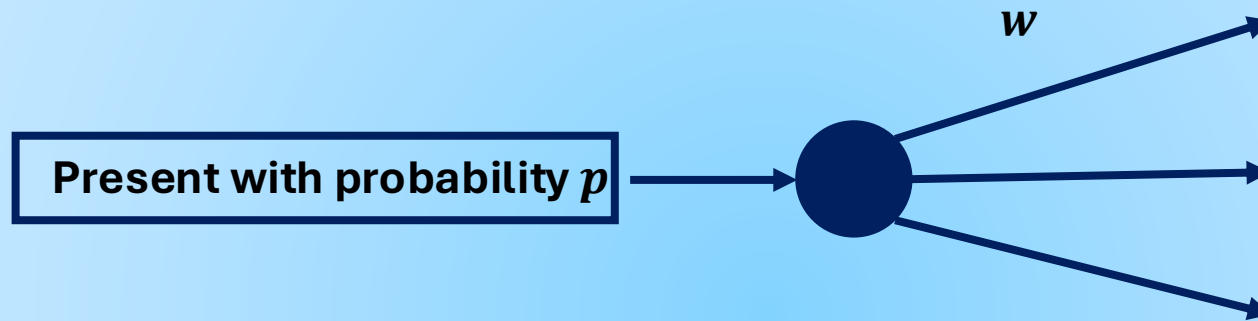
Introducing Dropout

A decorative graphic in the top right corner showing a small neural network with blue nodes and green connecting lines.

- Prevents Overfitting:
 - ✓ **approximately combining** exponentially many different neural network **architectures efficiently**
- How It Works:
 - ✓ **Randomly drop units** (hidden or input) **during training**
Temporarily **removes** them and their connections
 - ✓ Each training step uses a **different subnetwork**
Simulates an **ensemble of many models**

Dropout: Random Unit Selection

- Each unit is retained randomly with probability p

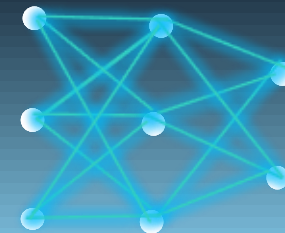
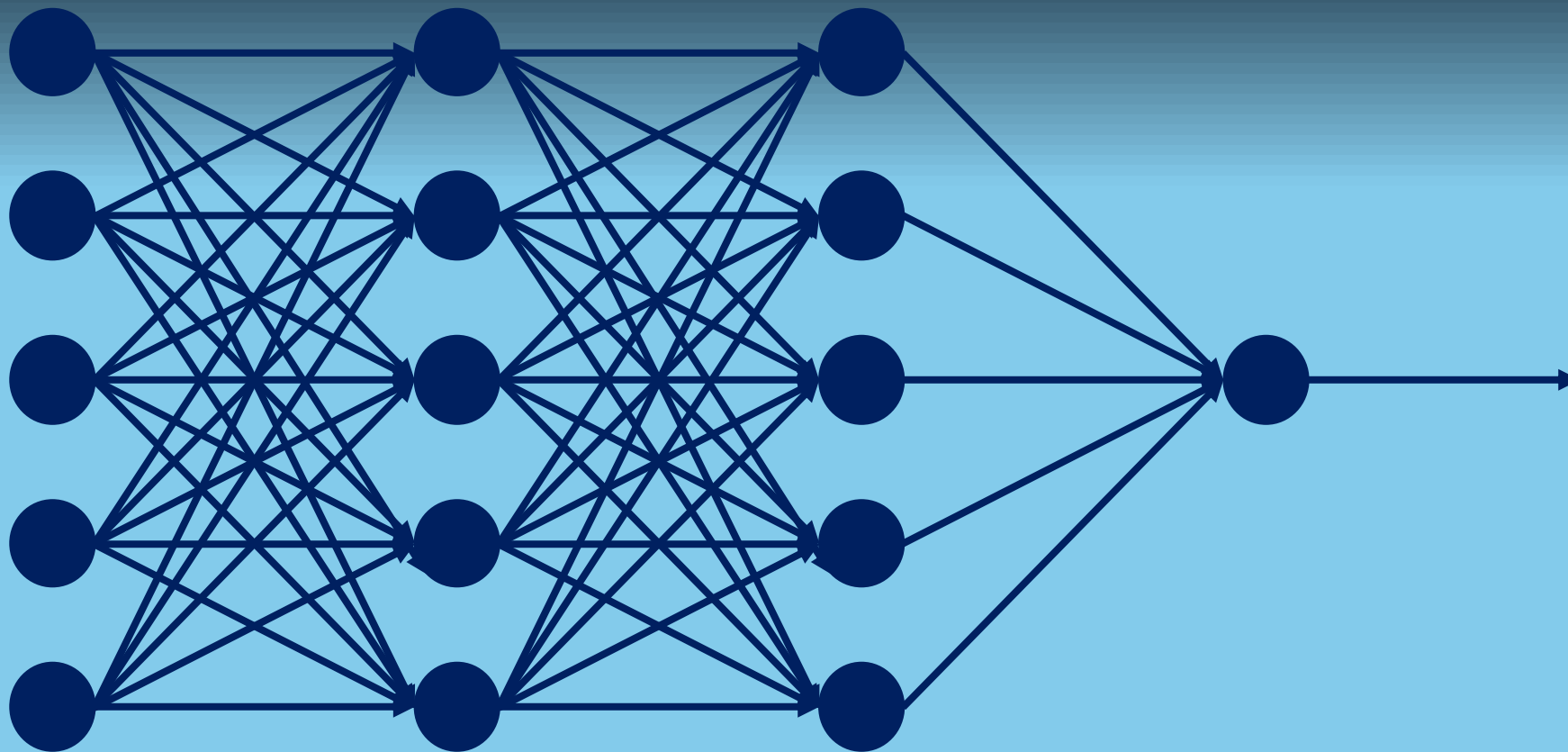


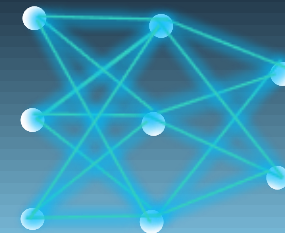
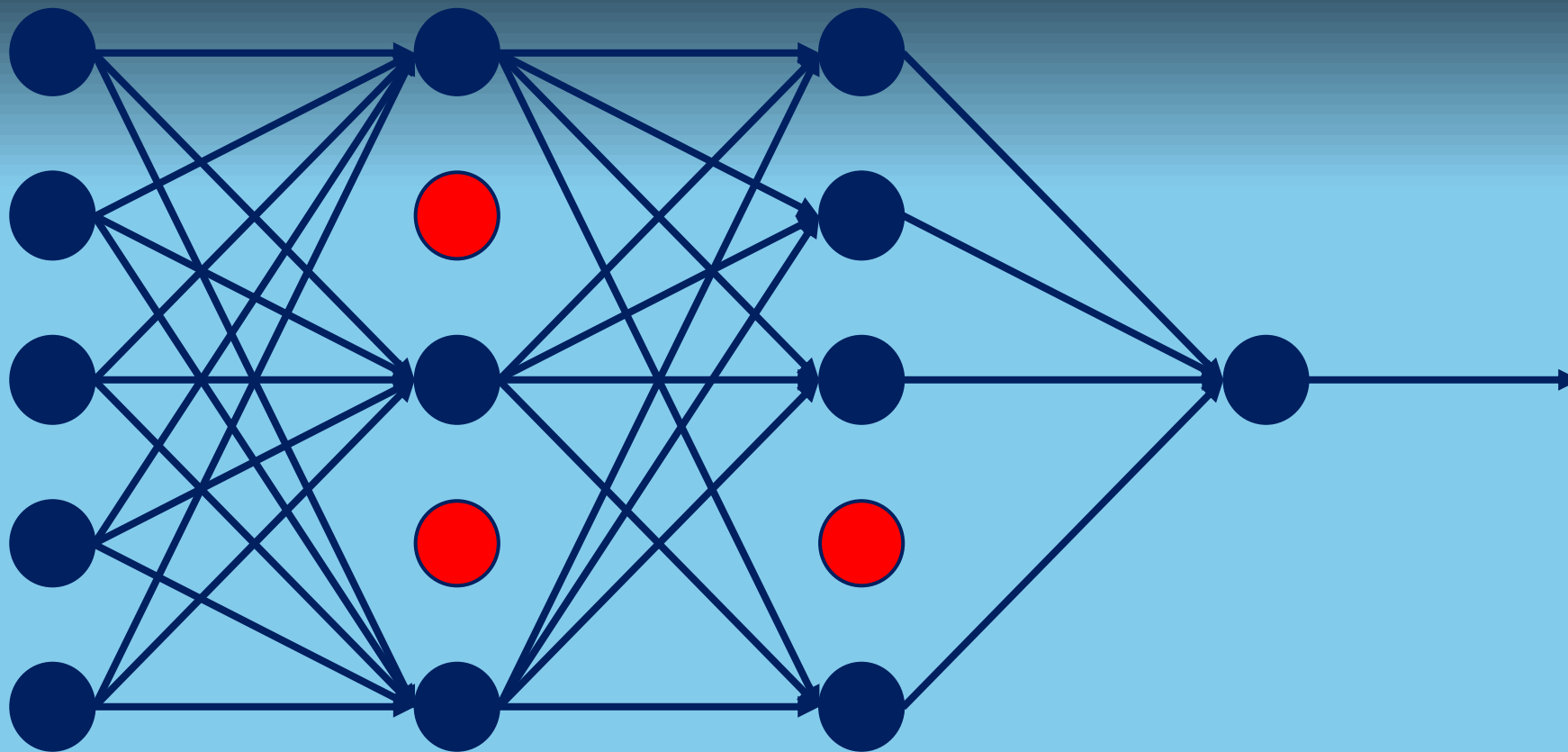
- p can be tuned using a validation set
- Dropping out 20% of the input units and 50% of the hidden units was often found to be optimal

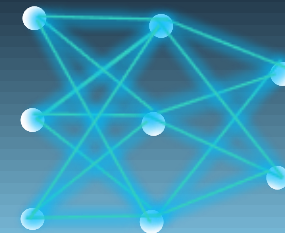
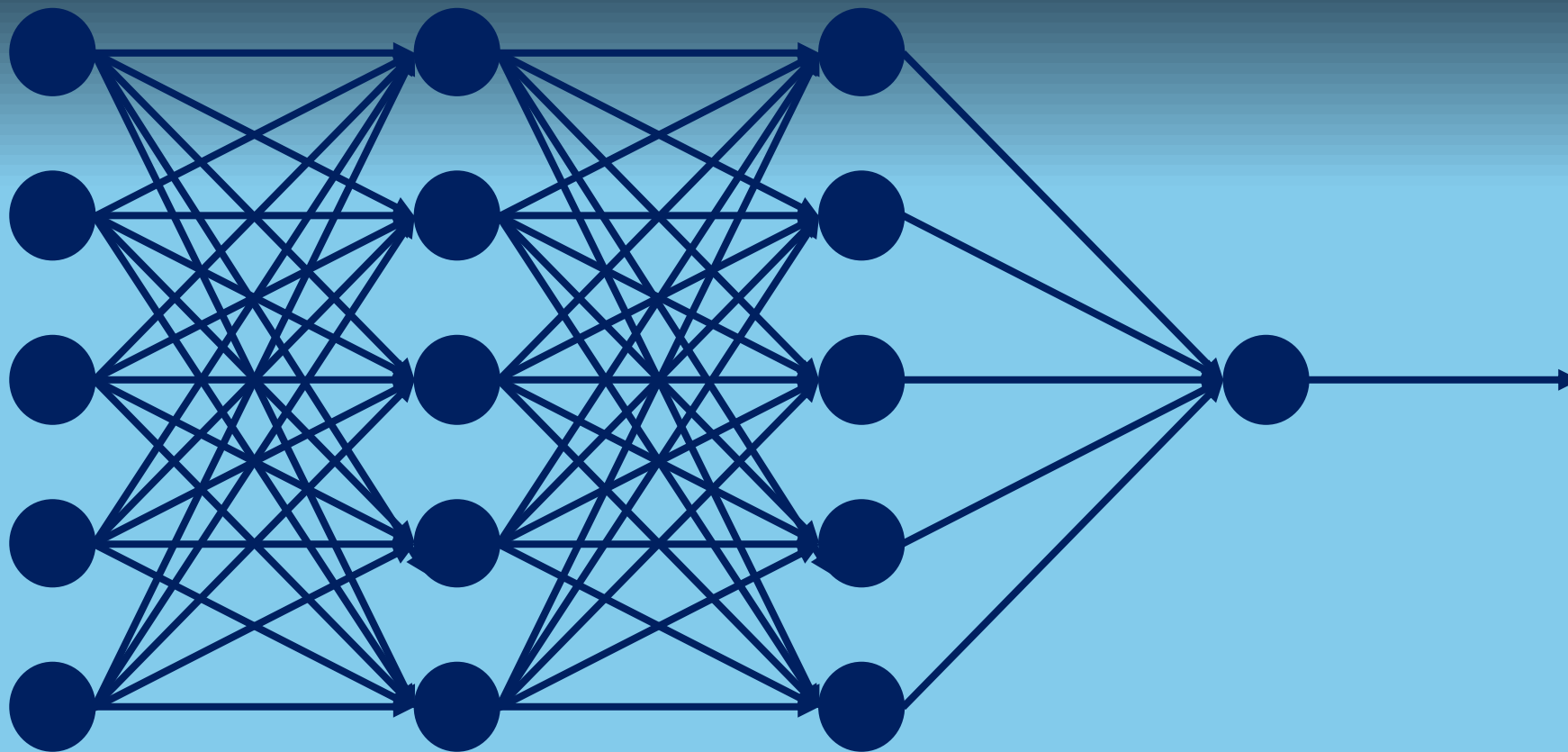
Dropout - Training Many Thinned Networks

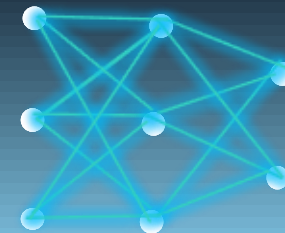
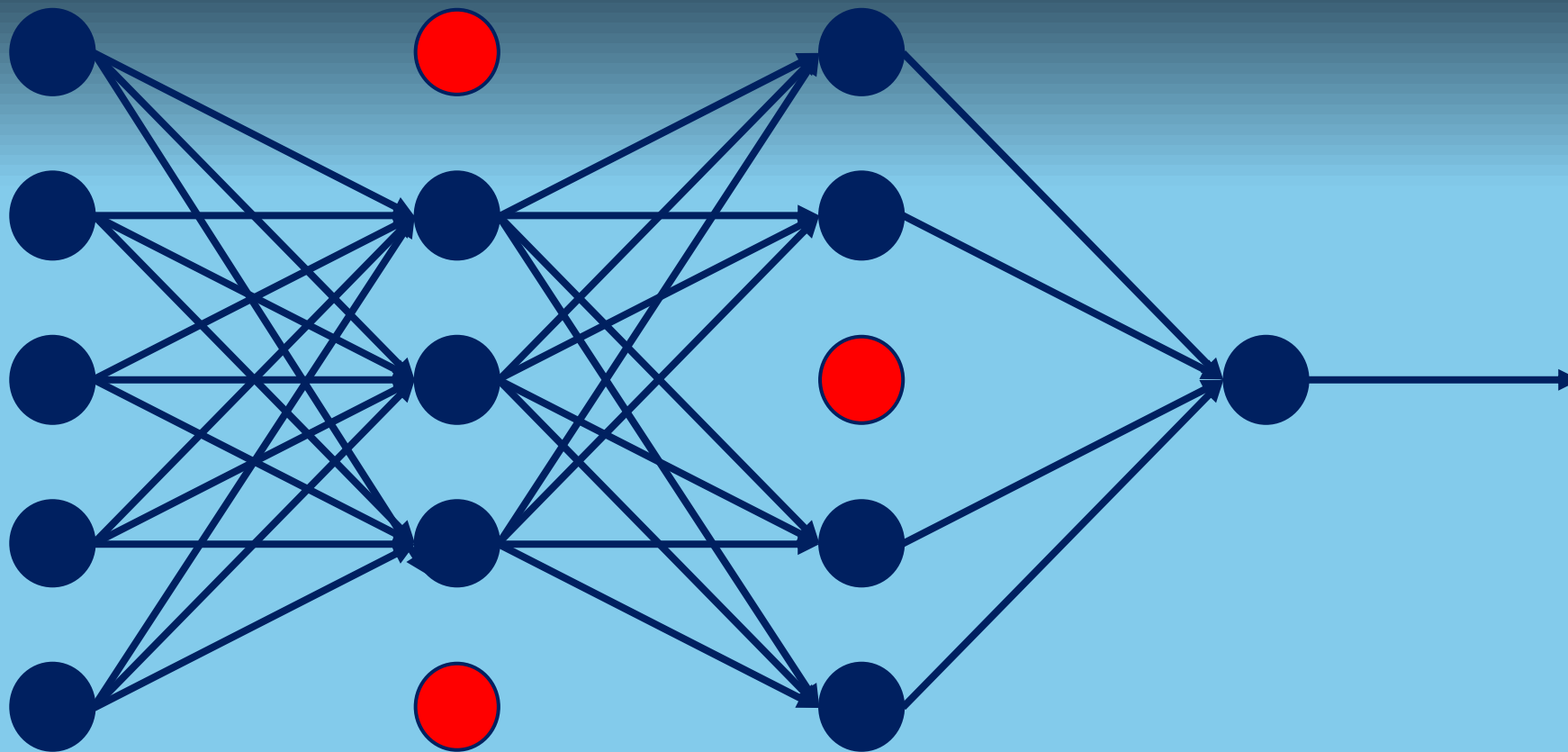


- Each training step samples a **random “thinned” subnetwork**. Keeps only a subset of units (those not dropped)
- A network with n units (excluding output units) represents 2^n **possible subnetworks**
- All subnetworks share weights
- Over training:
 - ✓ **Different subnetworks** are sampled **for each input**
 - ✓ Dropout effectively trains an ensemble of 2^n models
 - ✓ Each subnetwork is trained infrequently, if at all



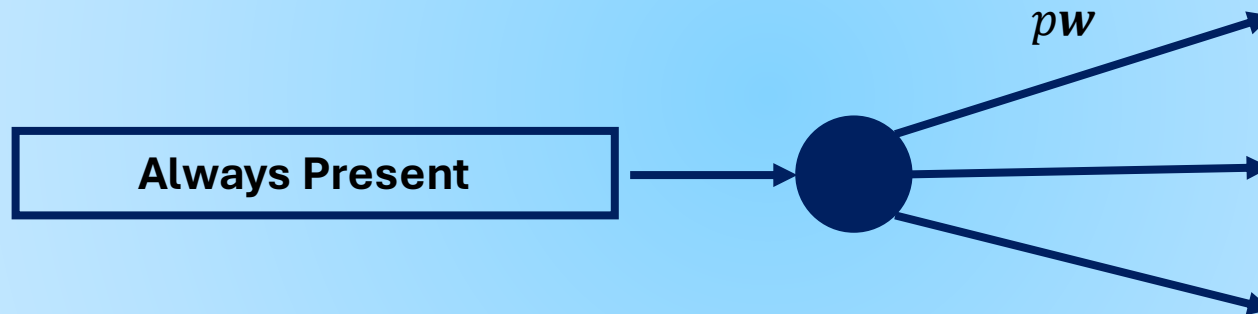






Dropout at Test Time

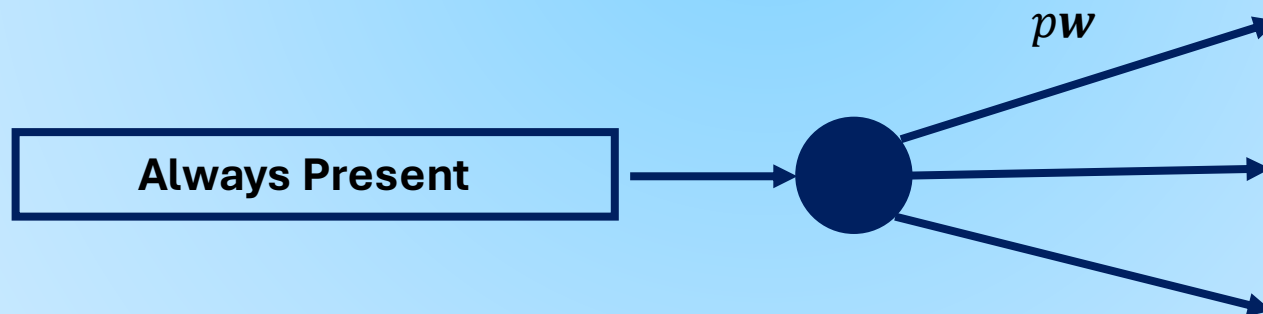
- **Averaging** over exponentially many subnetworks is **infeasible**
- Instead, use a **single deterministic network** with:
 - ✓ No dropout
 - ✓ Scaled weights



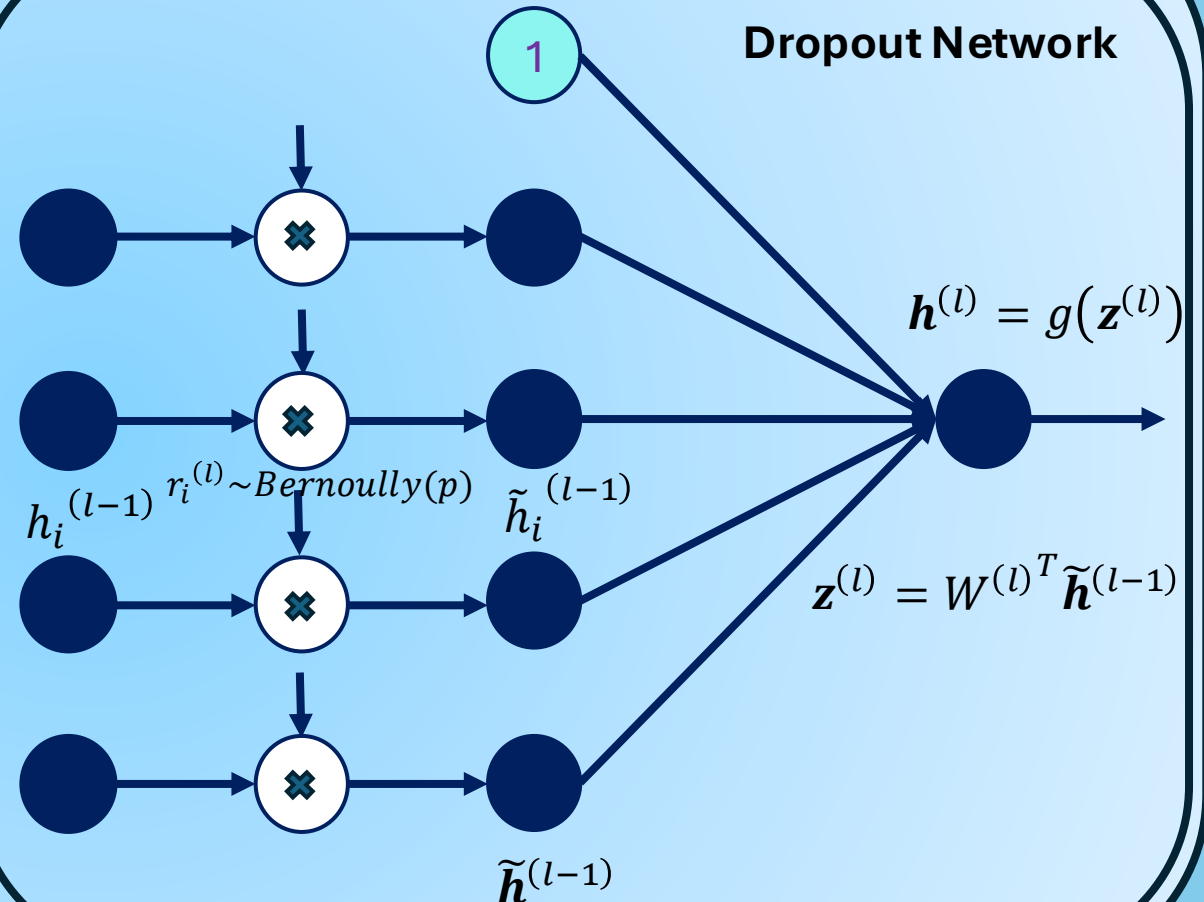
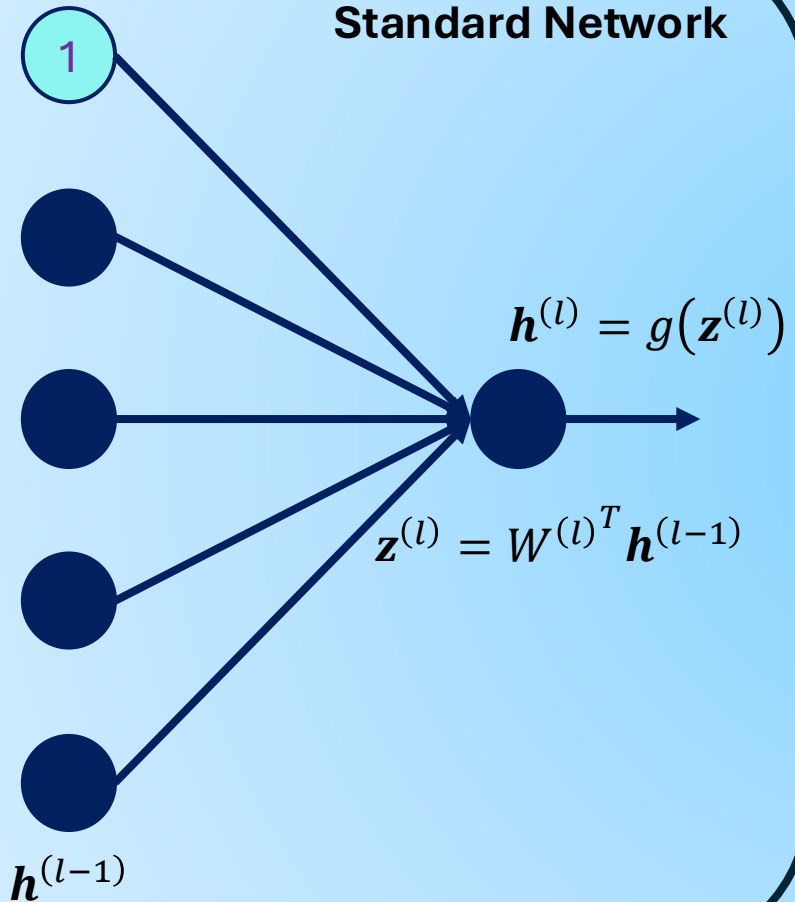
- **Weight Scaling Rule:**
 - ✓ If a unit was kept with probability p during training multiply its outgoing weights by p at test time

Dropout at Test Time

- Why This Works:
 - ✓ Ensures each unit's **expected output** under the distribution used to drop units at training time is the **same** as the **actual output at test time**
 - ✓ **Efficiently approximates** the effect of **combining** all 2^n subnetworks



Model Description



Dropout Improves Generalization

A decorative graphic in the top right corner of the slide, featuring a network of blue nodes connected by lines, resembling a neural network or a graph structure.

- **Training** a network with Dropout + **approximate model averaging** at **test time**:
 - Leads to significantly **lower generalization error**
- **Outperforms** many traditional regularization methods across a wide range of classification tasks

Learning Dropout Networks

A decorative graphic in the top right corner showing a small neural network with five nodes and several connecting lines.

- **Training** with Dropout uses **SGD**, like standard nets
- **Key difference:**
 - ✓ For **each training example** in a mini-batch, **sample a thinned network** by randomly dropping out units
 - ✓ **Forward and backpropagation** for that training case are **done** only on this **thinned network**

Evolutionary Motivation for Dropout

- Sexual reproduction breaks up co-adapted genes, forcing each gene to be useful on its own
- This promotes robustness and generalization through mixability
- Dropout mimics this by randomly removing units during training: Each unit must perform well without relying on others
- **Result:** Less co-adaptation, more independent, useful features

Co-Adaptation in Standard Neural Networks



- Each parameter **update depends** on what **all other** units are doing. Units may “fix” each other’s mistakes
- This leads to **complex co-adaptations**. Units **rely** on specific interactions with others
- **Problem:** These **co-adaptations** often **fail to generalize**. Resulting in **overfitting** on unseen data

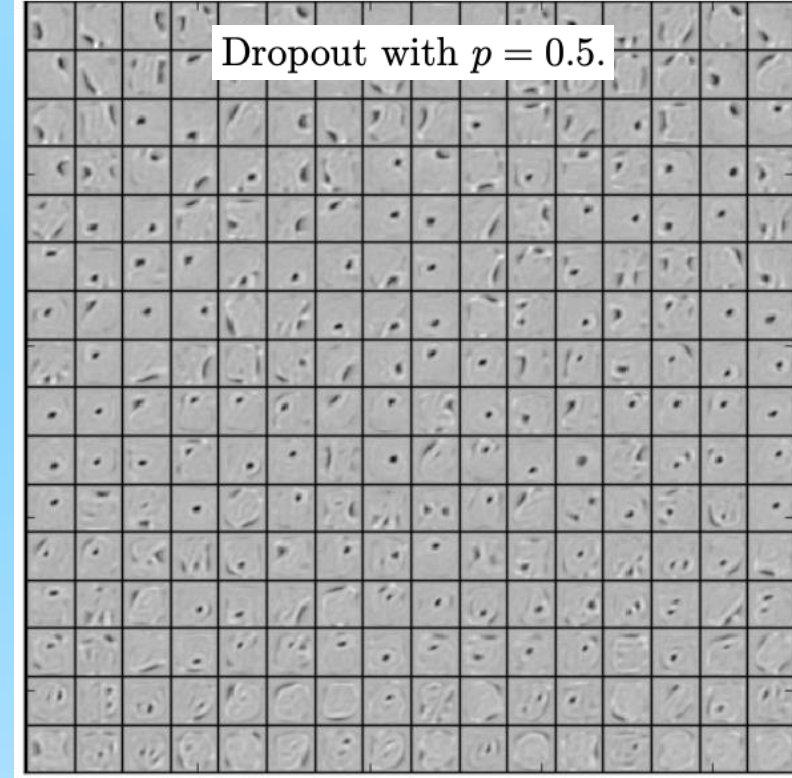
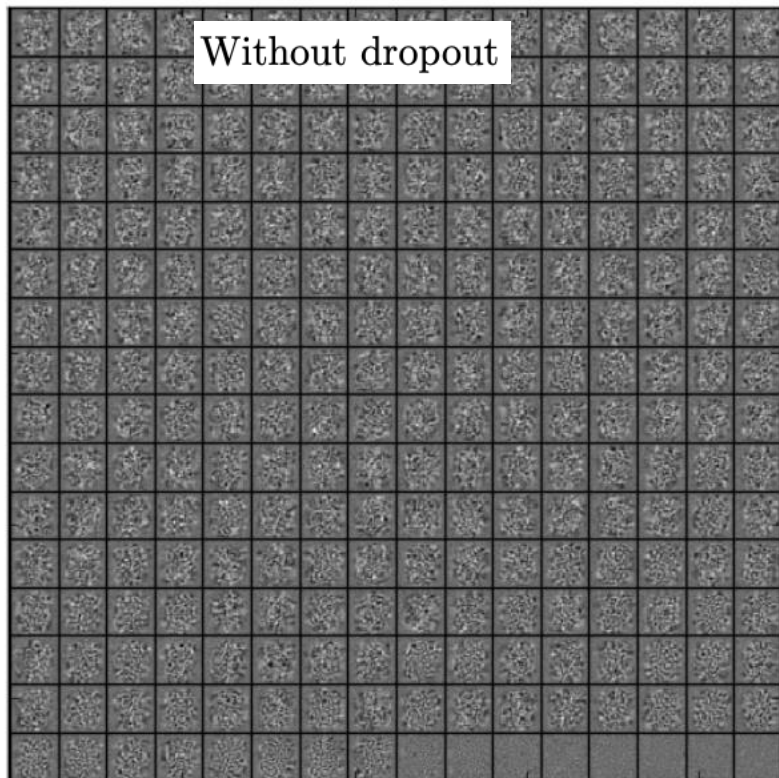
Dropout Prevents Co-Adaptation



- Key idea: **Dropout** makes other **units unreliable**
 - ✓ Each hidden unit cannot count on specific others being present
- As a result:
 - ✓ Units must be **individually useful**
 - ✓ They learn to **perform well across diverse combinations** of other units
- This **reduces co-adaptation**:
 - ✓ Promotes **robust**, generalizable features

Visualizing Dropout's Effect on Learned Features

- first-layer features learned with and without dropout.
- Task: Autoencoder on MNIST, 1 hidden layer, 256 ReLU units



Dropout Breaks Co-Adaptations in Practice



- **Without Dropout:**
 - ✓ Features are co-adapted and **lack individual meaning**
 - ✓ Hidden units **don't act as standalone detectors**
- **With Dropout:**
 - ✓ Units learn to **detect edges, strokes**, and localized features
 - ✓ Each unit **captures meaningful patterns** independently
- **Conclusion:**
 - ✓ Dropout disrupts co-adaptation
 - ✓ Leads to more **generalizable, interpretable** features

Alternative Implementation of Dropout

- **Standard dropout:**
 - ✓ Retain units with probability p during training
 - ✓ Scale weights by p at test time
- **Equivalent alternative (Inverted Dropout):**
 - ✓ At training time, scale retained activations by $1/p$
 - ✓ No scaling of weights needed at test time
- **Interpretation:**

Dropout = Multiply activation h_i by $r \sim \text{Bernoulli}(p) \times (1/p)$

$$E[r_\beta] = 1, \quad \text{Var}[r_\beta] = (1-p)/p$$

Generalizing Dropout: Gaussian Noise

- **Standard Dropout:**
Multiply activations by Bernoulli(p) random variables
- **Generalization:**
Use Gaussian noise instead: $r \sim N(1, \sigma^2)$
- **Key properties:**
 - ✓ Preserves the expected activation value
 - ✓ No weight scaling needed at test time
 - ✓ σ becomes a **tunable hyperparameter** (like p in Bernoulli dropout)
- **Empirical finding:**
Gaussian dropout often performs **as well as or better** than Bernoulli dropout

Training Time Trade-off with Dropout



- **Drawback:** Dropout increases training time
Typically 2–3 times longer than standard training
- **Why** training is slower:
 - ✓ Each training example effectively tries to train a different random architecture
 - ✓ parameter updates are very noisy
- But this **noise helps**:
 - ✓ Acts as a regularizer
 - ✓ Reduces overfitting by preventing co-adaptation
- Trade-off:
More training time → Higher dropout possible → Better generalization

Toward Faster Dropout: Marginalizing the Noise

A decorative graphic in the top right corner of the slide, featuring a network of blue nodes connected by thin, glowing blue lines, resembling a simplified neural network or a data flow diagram.

- Dropout = **stochastic regularization** using random noise (unit dropping). **Injecting noise** into hidden unit activations
- In dropout, we **minimize the loss** function **stochastically** under a noise distribution
- This **can be seen** as **minimizing an expected loss** function.

Deterministic Interpretation of Dropout

- Idea:
 - ✓ Get dropout's benefits **without stochasticity**
 - ✓ **Marginalize the noise** to derive an **equivalent deterministic regularizer**
 - ✓ These models:
 - ❖ Approximate dropout **without sampling**
 - ❖ **Require no random bits** during **training** or **inference**
 - ❖ **Allow exact gradients** of the marginalized loss function
 - ❖ In **simple** cases, dropout can be **analytically marginalized** out to obtain **deterministic regularization methods**
 - ❖ For **complex** models, deriving this regularizer is **non-trivial**

Dropout in Linear Regression

- **Linear Regression:**

$$\text{minimize}_w \|y - Xw\|^2$$

- **Dropout in Linear Regression:**

The input X is dropped out such that any input dimension is retained with probability p :

$$\tilde{X}_{i,j} = R_{i,j}X_{i,j} \text{ where } R_{i,j} \sim \text{Bernoulli}(p), \text{ iid}$$

$$E_R(\tilde{X}_{i,j}) = pX_{i,j}$$

$$E_R(\tilde{X}_{i,j}\tilde{X}_{k,l}) = (?)X_{i,j}X_{k,l}$$

Dropout in Linear Regression

- Linear Regression:

$$\text{minimize}_w \|y - Xw\|^2$$

- Dropout in Linear Regression:

The input X is dropped out such that any input dimension is retained with probability p :

$$\tilde{X}_{i,j} = R_{i,j}X_{i,j} \text{ where } R_{i,j} \sim^{iid} \text{Bernoulli}(p)$$

$$E_R(\tilde{X}_{i,j}) = pX_{i,j}$$
$$E_R(\tilde{X}_{i,j}\tilde{X}_{k,l}) = \begin{cases} p^2X_{i,j}X_{k,l} \\ pX_{i,j}^2 \end{cases}$$

Dropout in Linear Regression

$$\begin{aligned} & \text{minimize}_{\mathbf{w}} E_R \left(\|\mathbf{y} - \tilde{X}\mathbf{w}\|^2 \right) \\ & E_R \left(\|\mathbf{y} - \tilde{X}\mathbf{w}\|^2 \right) = \\ & E_R \left(\|\mathbf{y}\|^2 - 2\mathbf{y}^T \tilde{X}\mathbf{w} + \|\tilde{X}\mathbf{w}\|^2 \right) \\ & E_R \left(\|\mathbf{y}\|^2 - 2\mathbf{y}^T \tilde{X}\mathbf{w} + \mathbf{w}^T \tilde{X}^T \tilde{X} \mathbf{w} \right) = \\ & \|\mathbf{y}\|^2 - 2\mathbf{y}^T E_R(\tilde{X})\mathbf{w} + \mathbf{w}^T E_R(\tilde{X}^T \tilde{X})\mathbf{w} = \\ & \|\mathbf{y}\|^2 - 2\mathbf{y}^T (pX)\mathbf{w} + \mathbf{w}^T (p^2 X^T X + (p - p^2) \text{diag}(X^T X))\mathbf{w} = \\ & \|\mathbf{y}\|^2 - 2\mathbf{y}^T (pX)\mathbf{w} + \mathbf{w}^T ((pX)^T (pX))\mathbf{w} + p(1 - p)\mathbf{w}^T (\text{diag}(X^T X))\mathbf{w} = \\ & \|\mathbf{y} - pX\mathbf{w}\|^2 + p(1 - p)\|\Gamma\mathbf{w}\|^2 \\ & \Gamma = (\text{diag}(X^T X))^{1/2} \end{aligned}$$

Dropout in Linear Regression

- In linear regression, dropout is equivalent in expectation to a modified form of L2 regularization

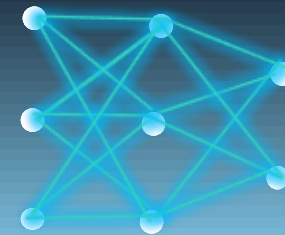
$$\text{minimize}_{\mathbf{w}} \|\mathbf{y} - pX\mathbf{w}\|^2 + p(1 - p)\|\Gamma\mathbf{w}\|^2$$

$$\Gamma = (\text{diag}(X^T X))^{1/2}$$

Dropout: Summary and Impact

A decorative graphic in the top right corner of the slide, consisting of several blue nodes connected by thin, glowing blue lines, resembling a simplified neural network or a star topology.

- **Problem:**
Standard training **encourages fragile co-adaptations** that don't generalize
- **Purpose:** **Reduce overfitting** and **improve generalization** in neural networks
- **Solution:**
 - ✓ Dropout randomly disables units during training
 - ✓ Forces units to work independently, improving robustness
- **Proven benefits across diverse domains.**



Thank you